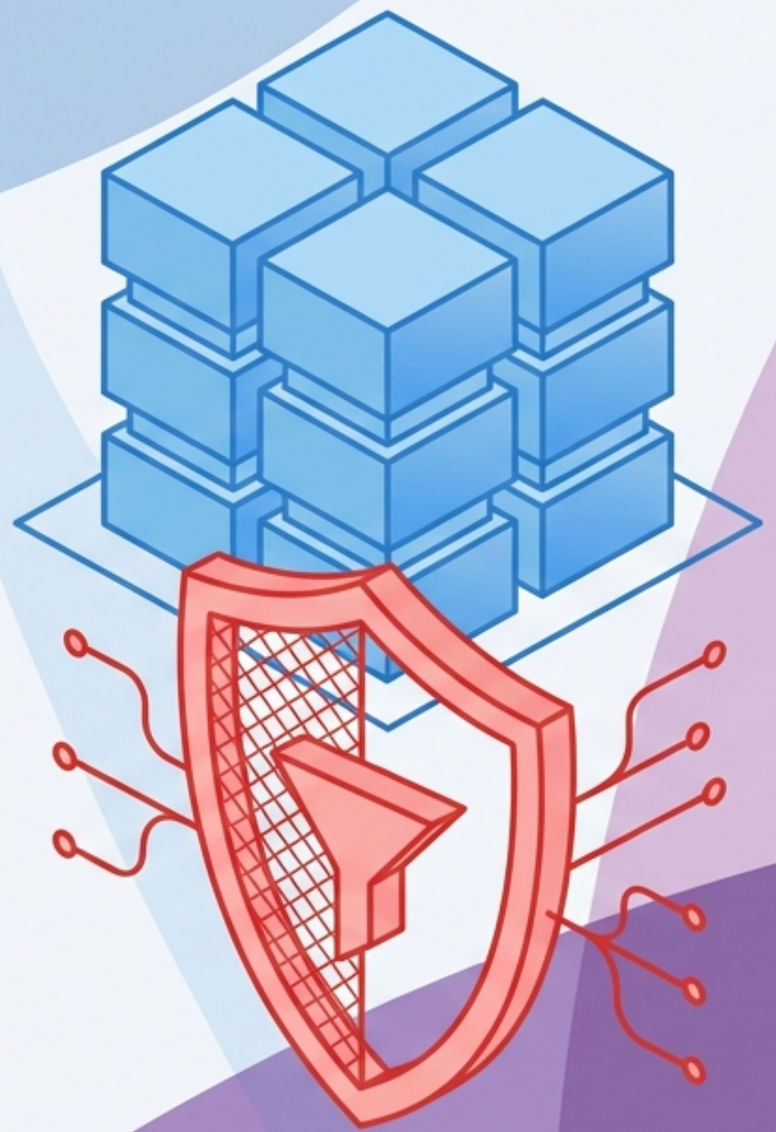
JavaScript Fundamentals

State & Stability

Mastering Client-Side Memory and Error Handling



Client-Side Technologies - Advanced JavaScript

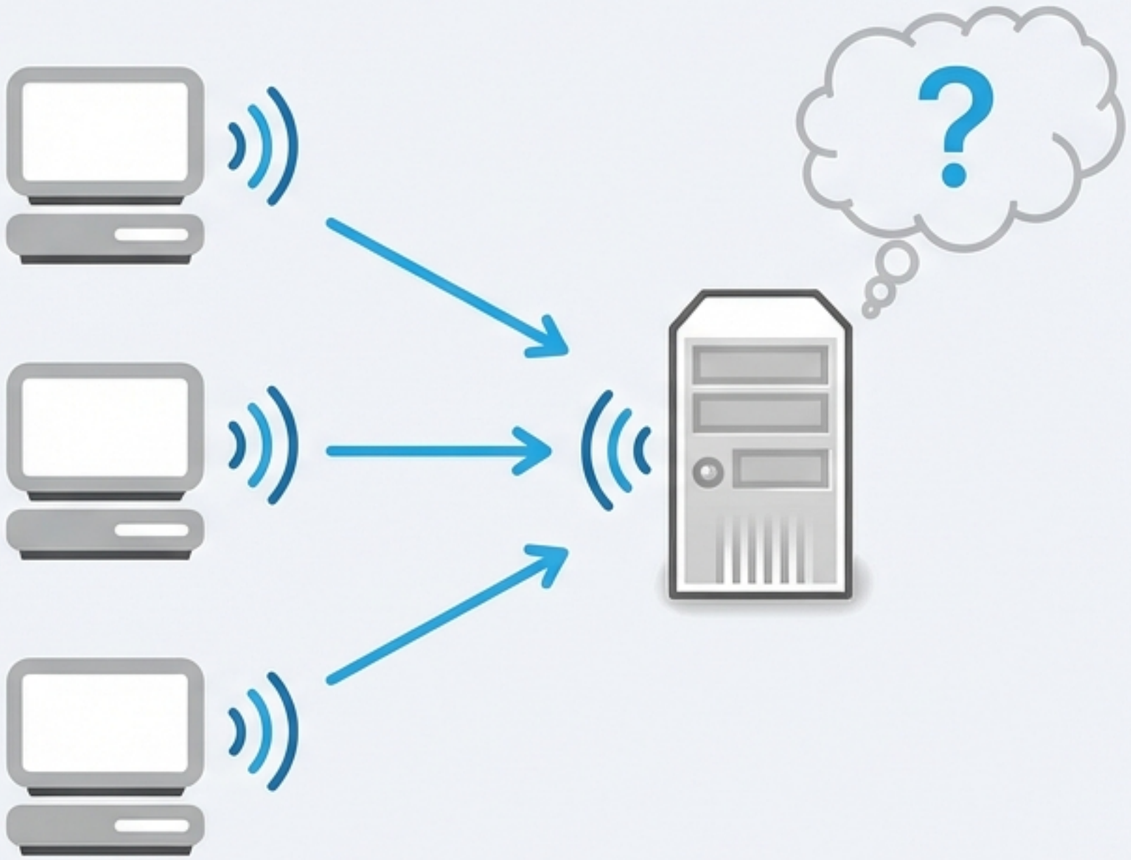
The Problem of the Stateless Web

HTTP is a stateless protocol. This means the server treats every request as a new interaction, forgetting the client immediately after the response is sent.

The Trade-off

- **Advantage:** Requires fewer server resources.
- **Disadvantage:** The client must provide context with every request.
- **Consequence:** Without “memory,” features like logins and shopping carts are impossible.

The Amnesiac Server



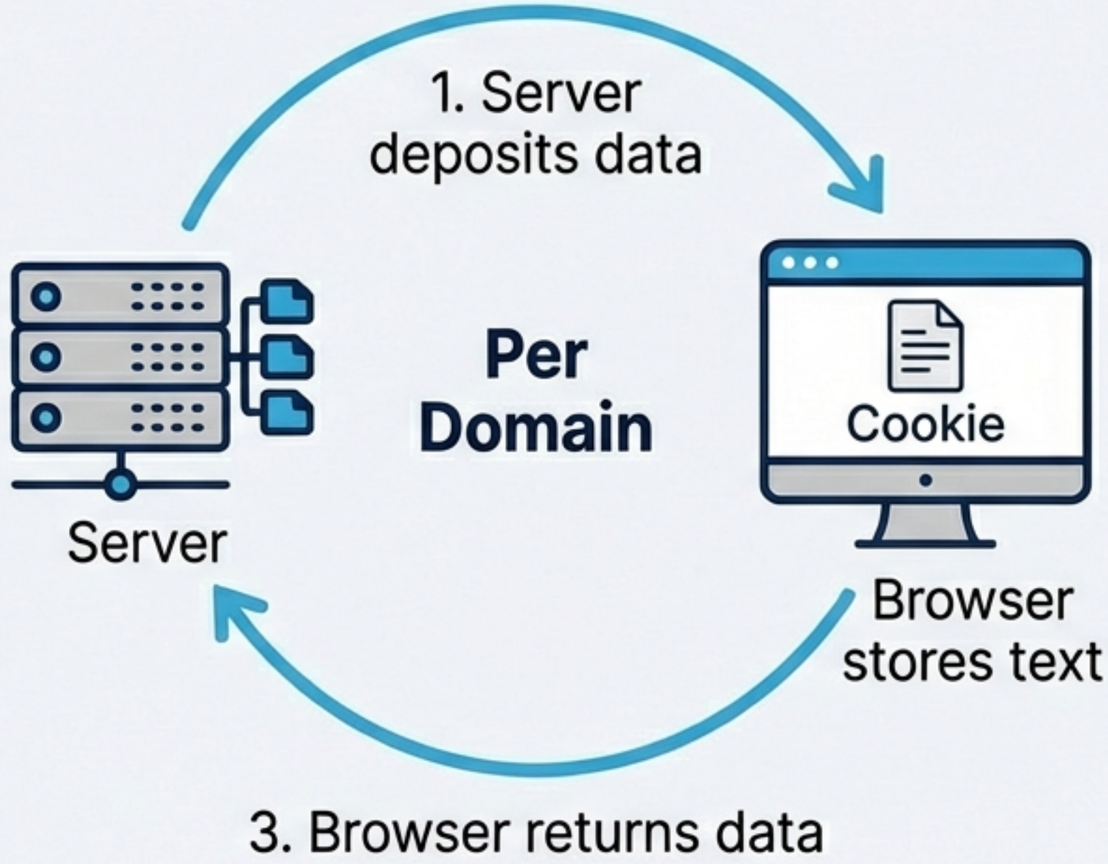
Curing Amnesia with Cookies

A mechanism for persistence

Cookies are small text strings stored on the client's machine. Invented by Netscape, they act as an identity card passed between client and server.

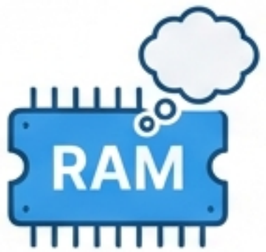
Important: A cookie is a passive text string, not a script or program.

Cookies were originally invented by Netscape to give 'memory' to web servers and browsers.



Cookie Taxonomy and Utility

Types of Cookies



Session Cookies

Reside in memory.
No expiry date.
Deleted when browser closes.

Persistent Cookies

Stored on hard drive.
Have an expiry date.
Survive browser restarts.

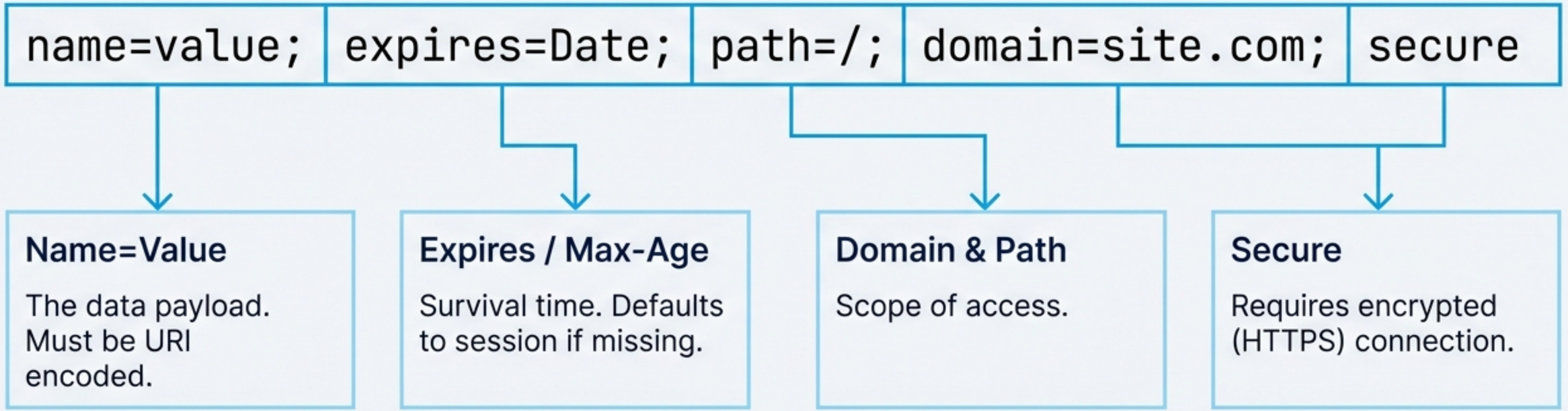
Common Use Cases

- **Authentication:** Avoiding re-login.
- **State Management:** Shopping carts.
- **Personalization:** Themes & preferences.
- **Tracking:** Analyzing user behavior.

The Anatomy of a Cookie

Data Structure breakdown

Parameter	Description
`name=value`	Data payload (URI encoded)
`expires=Date`	Survival time (session default)
`path=/`	Scope of access (URL path)
`domain=site.com`	Scope of access (server domain)
`secure`	Encrypted connection only



Implementing Cookies in Code

Writing to document.cookie

```
var myDate = new Date();
myDate.setDate(myDate.getDate() + 7); // Set for 1 week

// Setting a cookie requires URI encoding
document.cookie = 'username=' + encodeURIComponent('JohnDoe') +
    ';expires=' + myDate.toGMTString();
```

Crucial for handling special characters, whitespace, and semicolons.

Assigning a string here adds or updates a specifc cookie; it does not overwrite all other cookies.

Retrieval and Deletion Logic

Reading Cookies

`document.cookie` returns a single string of all cookies separated by semicolons.

Deleting Cookies

There is no `delete` command. You must overwrite the cookie with a past expiry date.

```
document.cookie = 'user=; expires=Thu, 01 Jan 1970 00:00:00 UTC;';
```

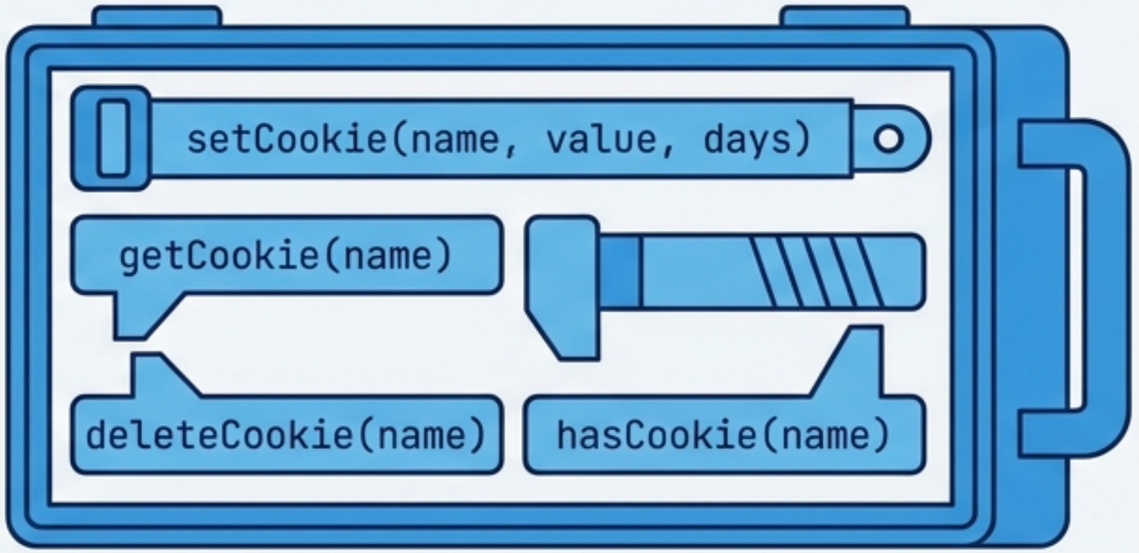
The Need for Abstraction

Creating a Cookie Function Library

Problem

Raw string manipulation is tedious and error-prone. Date math is repetitive.

Solution



Abstracting the complexity into reusable functions ensures consistency and reduces bugs.

Limitations & Security Realities

Hard Limits

4KB Max Size per Cookie

20 Cookies per Domain

300 Total Cookies in Browser

Security Context

Plain Text: Readable by anyone. Never store sensitive secrets.

Same-Origin Policy: Domains cannot read each other's cookies.

Dispelling Myths

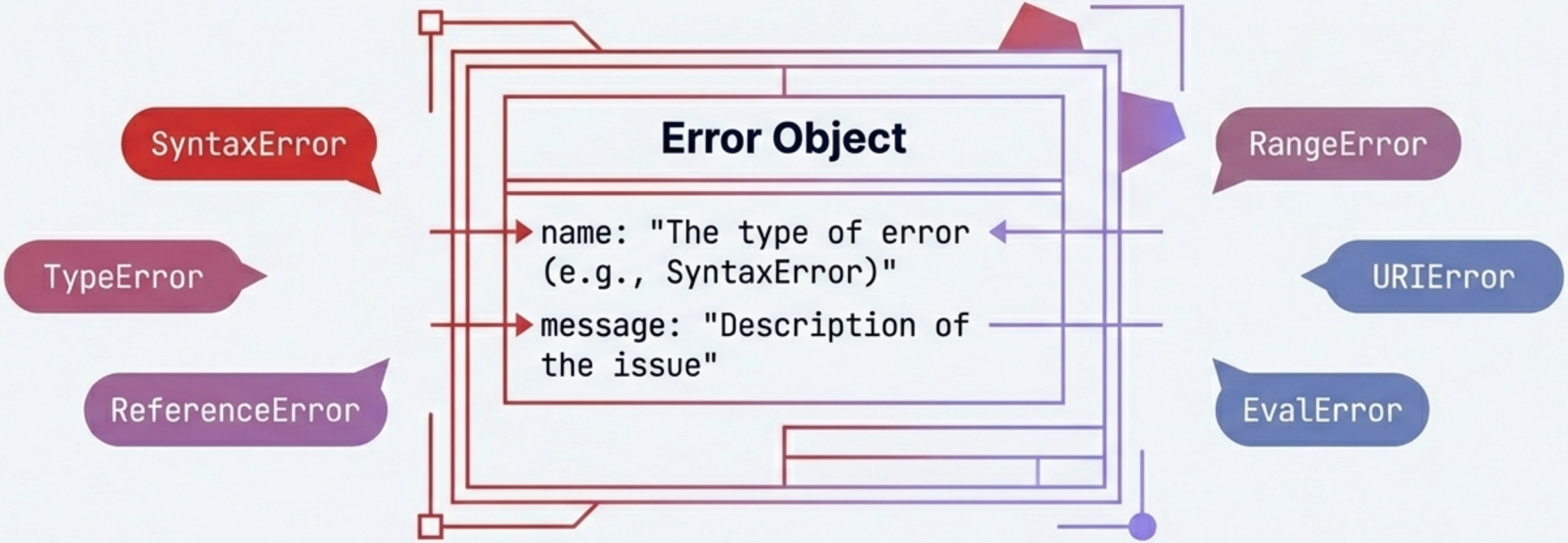
Can erase hard drives **(False)**

Transmit viruses **(False)**

Generate pop-ups **(False)**

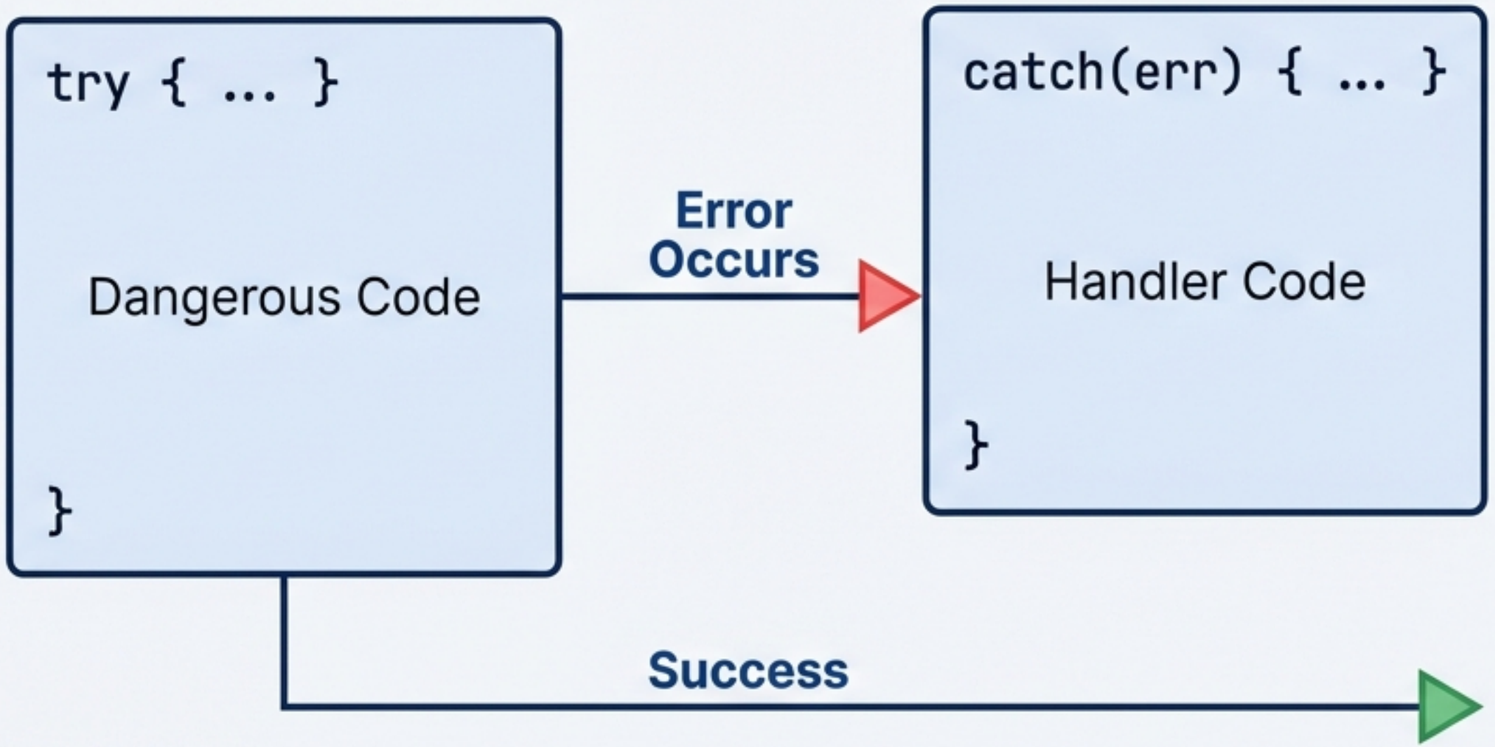
Managing Application Fragility

Introduction to the Error Object



Errors are created implicitly by the browser or explicitly via “new Error()”.

The Defensive Shield: try...catch

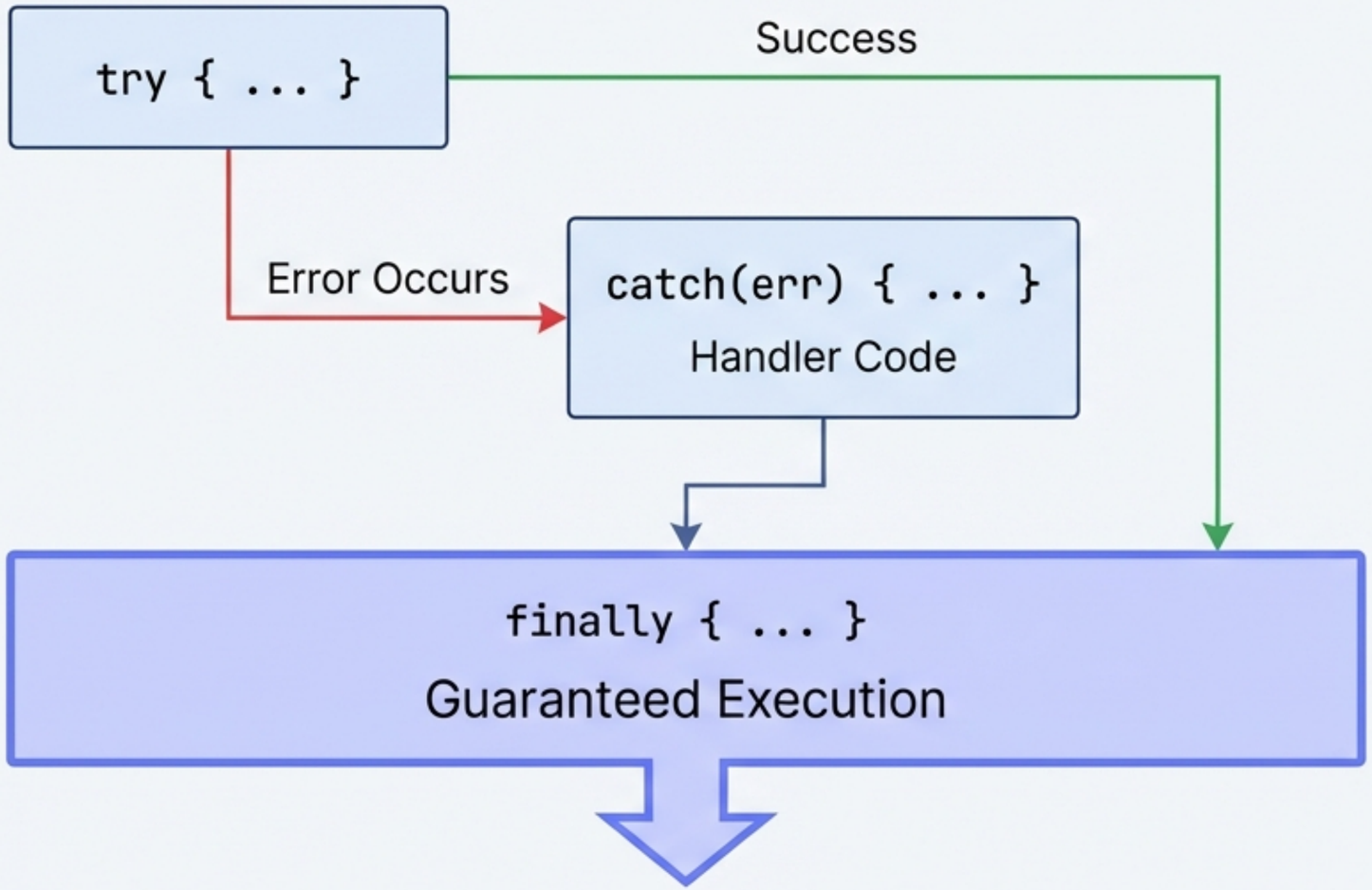


Critical Constraint

Asynchronous Gap: try...catch does not catch errors inside asynchronous callbacks (like setTimeout) implicitly.

```
try {
  setTimeout(() => {
    throw new Error('Async error');
  }, 1000);
} catch (err) {
  console.log(err);
  // Does not catch the error
}
```

Ensuring Execution with 'finally'



The `finally` block executes regardless of the outcome. Ideal for cleanup tasks like closing streams or hiding loading spinners.

Taking Control with ‘throw’

Enforcing Business Logic

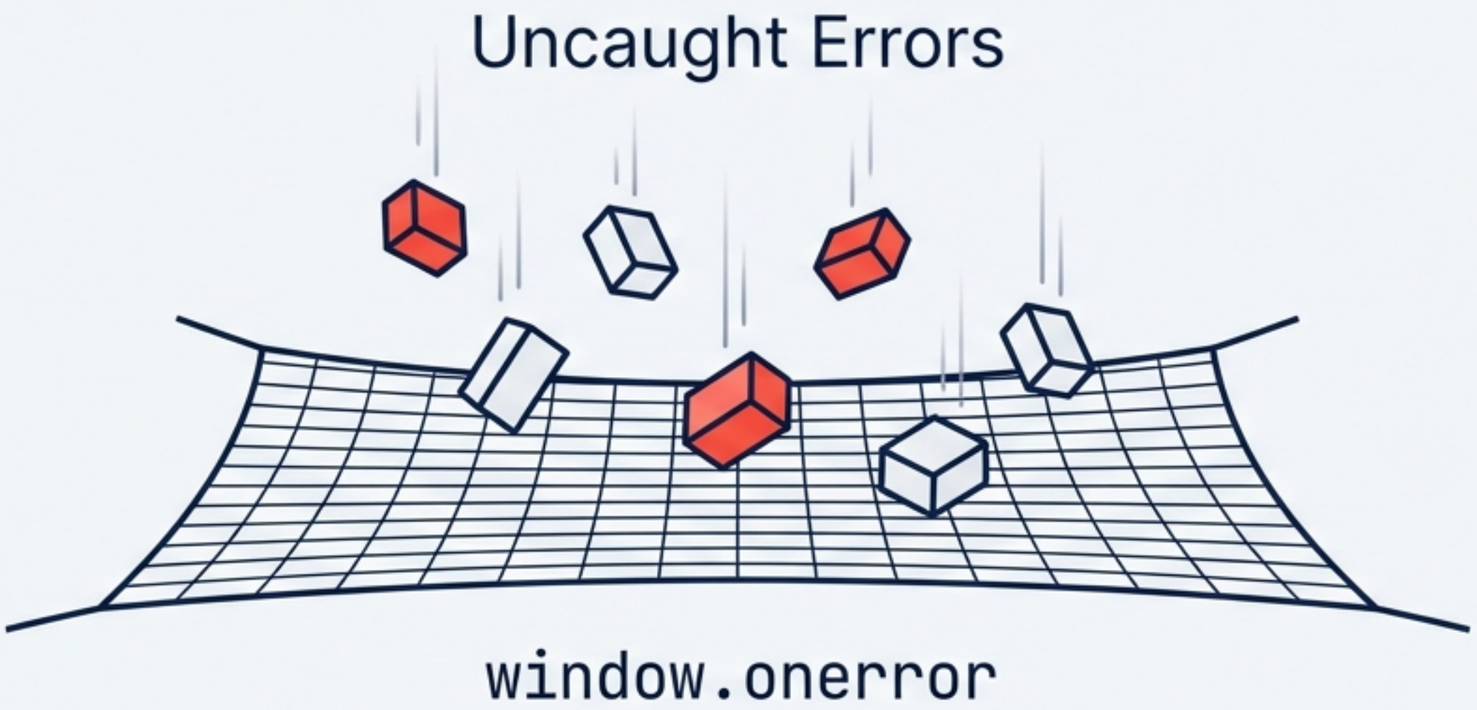
The browser catches syntax errors.
You must throw business errors.

Generates a custom exception. Can be a String, Number, Boolean, or Object.

```
try {
  if(x > 100) {
    throw 'Error: Value too high';
  }
} catch(err) {
  // err is 'Error: Value too high'
  alert(err);
}
```

The Last Resort: window.onerror

A global event handler for errors missed by `try...catch` blocks.



Return Value: returning "`true`" prevents the browser from firing the default error message (suppression).

1. Message
2. URL
3. Line Number
4. Column
5. Error Object

Foundations of Robust Development

Summary & Modern Context

State Management

- Use Cookies for small, persistent data authentication.
- Respect the 4KB limit.
- Abstract complexity with helper functions.
- Modern Alternative: Web Storage API (localStorage).

Application Stability

- Wrap risky code in try...catch.
- Use 'throw' to enforce logic rules.
- Use 'finally' for cleanup.
- Fail gracefully to protect user experience.

A robust application remembers its user's context and withstands its own internal errors.